# PDF Generator -- Mentoring Guide

A Streamlit app that builds a real PDF file from user-entered content blocks, entirely in memory, and offers it back as a download. This guide walks through the moving pieces, then a real bug this project's own build surfaced, then closes with exercises.

## 1. Two different PDF-building jobs in one project

This app actually uses reportlab *twice*, for two unrelated purposes: `app.py` builds whatever PDF the *user* designs through the UI, and this very script, `build_guide.py`, builds the mentoring guide PDF you're reading right now. Same library, same underlying API (`SimpleDocTemplate` + a list of "flowable" elements like `Paragraph` and `ListFlowable`) -- worth noticing since it means reading this guide's own source is a second, real example of everything Section 3 explains.

## 2. Building a PDF in memory: io.BytesIO

reportlab's `SimpleDocTemplate` normally writes to a file path on disk. This app never touches disk at all -- it hands `SimpleDocTemplate` an `io.BytesIO()` object instead, which behaves like a file (it has `.write()`) but just holds bytes in memory:

```
buffer = io.BytesIO()
doc = SimpleDocTemplate(buffer, pagesize=A4, title=title)
doc.build(story)
return buffer.getvalue()   # the raw PDF bytes
```

This matters on a server: temp files need cleanup, can collide between concurrent users, and are slower than memory. `io.BytesIO` sidesteps all of that -- the PDF exists only as bytes in a Python variable, for exactly as long as this function call needs it.

## 3. reportlab's model: a list of flowables

reportlab doesn't think in terms of x/y coordinates for a document like this. You build a Python list -- called the *story* -- of "flowable" objects (`Paragraph`, `Spacer`, `ListFlowable`), and `doc.build(story)` lays them out top to bottom, wrapping text and adding page breaks automatically.

```
story = [Paragraph(title, styles["Title"])]
for block in blocks:
    if block["type"] == "Heading":
        story.append(Paragraph(block["text"], styles["Heading2"]))
    elif block["type"] == "Bullet list":
        items = [line.strip() for line in block["text"].splitlines() if line.strip()]
        story.append(ListFlowable([ListItem(Paragraph(i, styles["Normal"])) for i in items]))
    else:
        story.append(Paragraph(block["text"], styles["Normal"]))
```

`getSampleStyleSheet()` is a ready-made set of named paragraph styles (`"Title"`, `"Heading2"`, `"Normal"`) -- reused as-is here rather than hand-building fonts and sizes.

## 4. Serving the bytes: st.download_button

`st.download_button` takes the finished bytes directly and hands them to the browser as a file download -- no separate route or static file needed, unlike a typical web framework:

```
st.download_button(
    "■■ Download PDF",
    data=pdf_bytes,
    file_name=f"{doc_title}.pdf",
    mime="application/pdf",
)
```

Notice the PDF is rebuilt from `st.session_state.blocks` on *every* rerun, not just when a "Generate" button is clicked -- cheap enough for a document this size, and it means the download button always offers the current state of the document, with no extra "did you forget to regenerate" step for the user to trip on.

## 5. A real bug this project's own build found: widget identity inside a form

The block-type selector offers three kinds of content (`Paragraph`, `Bullet list`, `Heading`), and the first version of this app tried to change the text box's label depending on which one was picked -- "One item per line" for a bullet list, something more generic otherwise:

```
# THE BUGGY VERSION -- do not copy this
with st.form("add_block_form", clear_on_submit=True):
    block_type = st.selectbox("Block type", BLOCK_TYPES)
    if block_type == "Bullet list":
        text = st.text_area("One item per line")
    else:
        text = st.text_area("Text")
```

This looks reasonable, and would work fine *outside* a form. Inside `st.form(...)`, though, widgets deliberately don't trigger a rerun when a sibling field changes -- that's the whole point of a form, batching input until submit. So in the browser, picking "Bullet list" from the selectbox never actually reruns the script, which means the text box the user is typing into is still the one from whichever branch rendered on the *previous* run.

The real damage happens at submit time: the script finally reruns, now sees `block_type == "Bullet list"`, and executes `st.text_area("One item per line")` for the first time this run. Because neither call had an explicit `key=`, Streamlit auto-derives one partly from the label -- and a *different label* means a *different widget identity* as far as Streamlit is concerned. The text the user had already typed, stored under the old label's auto-key, doesn't transfer. It's silently discarded, and `text` comes back empty.

This was caught automatically: an automated test that filled in the form and submitted a bullet-list block got back an empty `st.session_state.blocks` where it expected one new entry -- exactly the silent-data-loss signature this class of bug produces, with no exception or error message anywhere.

The fix in the shipped `app.py`: one single, statically-labelled `st.text_area`, given an explicit fixed `key="block_text"`, so its identity never depends on anything that can change mid-form:

```
text = st.text_area(
    "Text",
    placeholder="A paragraph or heading's text, or one bullet item per line for a bullet list",
    key="block_text",
)
```

The general lesson, useful well beyond this app: **inside a st.form, never let a widget's label, or anything else that influences its auto-generated key, depend on another field in that same form.** Give it an explicit, fixed `key=` instead, and keep any value-dependent text (a caption, a hint)

separate from the widget itself.

## 6. Exercises

- **Reorder blocks.** Add ↑/↓ buttons next to each block that swap it with its neighbour in `st.session_state.blocks` (same index-based mutation pattern as the delete button).
- **Edit in place.** Let the user click a block to edit its text, instead of only being able to delete and re-add it.
- **Images.** Add an `st.file_uploader` for an image, and a fourth block type that embeds it (reportlab's `platypus.Image` flowable, fed the uploaded file's bytes via `io.BytesIO` -- the same in-memory-file idea as the PDF itself).
- **Page numbers.** reportlab supports a page-template callback for headers/footers -- add page numbers to the generated document.
- **Save/load a draft.** Persist `st.session_state.blocks` to a local JSON file (same pattern as the to-do list and expense tracker projects), so a half-built document survives closing the browser tab.

*Written as a teaching companion to app.py -- read this guide with the code open side by side.*